READING SHEET

Event Handling & Deployment

Module 3 · Frontend: React & Next.js · Day 48 · Week 10

PROGRAM	WORK TYPE	EST. TIME
CodeOps · Full Stack	**Revision + mini-project**	**2–3 hours**

> **Tip How to use this sheet**
> Two halves. Parts 1 and 2 cover the event system you have been using since Day 28 and how to build interaction that works with a keyboard and a thumb as well as a mouse. Parts 3 and 4 take your application to a URL somebody else can open. Deploy your capstone tonight — tomorrow is a build day, not a first-deployment day.

Part 1 · The Synthetic Event System

`onClick` is not a DOM listener on that element. React attaches **one listener at the root** and works out which component you meant — event delegation, which Module 2 taught you to write by hand, done for every event in the application. That is why a thousand rows cost nothing to make clickable.

YOU WRITE	WHAT HAPPENS
onClick on a button	No listener on that button
A hundred rows with onClick	Still one listener, at the root
event.target	The real DOM element
The event object	A wrapper over the native one
onChange on an input	Actually the native input event

The names differ slightly from plain HTML: camelCase rather than lowercase, `onChange` firing on every keystroke rather than on blur, and the handler being a function rather than a string.

The event object

```
                                                    the parts you will actually use
event.preventDefault();    // stop the browser default
event.stopPropagation();   // stop it reaching parents

event.target;              // where it happened
event.currentTarget;       // where the handler is
event.nativeEvent;         // the real DOM event
```

A click on the icon *inside* a button gives you the icon as target and the button as currentTarget — reading the wrong one is a bug that appears only when someone clicks a particular pixel. Note that event pooling is gone: older React reused event objects, so old advice about `event.persist()` no longer applies.

Stopping things, carefully

```
                                                    a clickable card containing a button
<div onClick={openDish}>
  <h3>Doro Wat</h3>
  <button onClick={(e) => {
    e.stopPropagation();     // or the card opens too
    addToCart();
  }}>Add</button>
</div>
```

> **Warning A clickable div is a warning sign**
> It is not focusable, not keyboard-reachable and not announced as interactive. If it does something it should be a button or a link — and then this nesting problem mostly disappears. Use stopPropagation sparingly in any case: it makes behaviour depend on invisible ancestors. And preventDefault and stopPropagation are not the same thing; neither implies the other.

Handlers that behave

PROBLEM	SYMPTOM	FIX
Handler called immediately	Runs on render	onClick={() => f(id)}
Stale value in the handler	Reads old state	The Day 28 functional updater
Double submit	Two orders, one click	Disable while pending
Rapid repeated input	A request per keystroke	Debounce — Day 41
Scroll or resize handler	The page stutters	Throttle, clean up on unmount

Everything there is a nuisance except the **double submit**. An impatient tap on a slow connection places two orders and the customer sees one — disable the button while the action is pending, and make the write idempotent on the server so a repeat cannot charge twice.

Part 2 · Keyboard, Mouse & Touch

Most of your users are on a phone and some are on a keyboard. Building for the mouse alone excludes both, and the fixes are small if you make them now rather than after the capstone.

```
                                                    the keys you will handle
function onKeyDown(e) {
  if (e.key === "Escape") close();
  if (e.key === "Enter" && !e.shiftKey) submit();
  if (e.key === "ArrowDown") moveSelection(1);
}
// e.key is the character or a name — never use keyCode
```

ELEMENT	YOU GET FOR FREE
`button`	Focus, Enter, Space, and announcement
`a href`	Focus, Enter, and middle-click
`div onClick`	Nothing at all

> **Warning The right element is the fix**
> Almost every keyboard problem in a React application is a div doing a button's job. Change the element and the handling, the focus ring and the announcement all arrive for nothing. And never remove the focus ring: outline: none with no replacement makes an application unusable by keyboard.

Touch

ON A PHONE	WHAT TO DO
No hover state exists	Never hide anything behind hover
Fingers are imprecise	Targets at least 44 by 44 pixels
Targets too close together	Space them, or taps go astray
A tap has a slight delay	Give immediate visual feedback
The keyboard covers the screen	Scroll the focused field into view

Hover is the common failure: a menu that appears on hover simply *does not exist* on a phone. If most of your users order from a phone, that is most of your users — so test on a real device rather than a narrow browser window. `onPointerDown` fires for mouse, touch and stylus alike, so use it for anything you would otherwise write twice.

Forms, once more

```
                                                    the events a form actually has
<form onSubmit={handleSubmit}>        // not onClick
  <input type="tel" inputMode="numeric"
         onChange={(e) => setPhone(e.target.value)}
         onBlur={validate} />
  <button disabled={pending}>
    {pending ? "Placing..." : "Place order"}</button>
</form>
```

A form submitted with Enter never touches the button, so handling the click instead means the keyboard path silently does nothing — a bug nobody using a mouse will ever find. And `type="tel"` gives a TeleBirr number the numeric keypad instead of a full keyboard: one attribute, and the form becomes noticeably easier to fill.

Interaction you can trust

RULE	WHY
Real buttons and links	Keyboard, focus and semantics free
Escape closes anything modal	Everyone expects it
Focus moves into a dialog	And returns when it closes
Feedback within 100ms	Otherwise the tap felt ignored
Nothing hidden behind hover	Half your users have no pointer

Put the mouse away and reach every function with Tab, Enter and Escape. Five minutes finds more real problems than any automated audit — then repeat it on an actual phone.

Part 3 · Preparing to Deploy

Most deployment failures are not deployment problems. They are things that were always broken and only became visible once the code left the machine it was written on.

```
                                                    the two commands that find most problems
npm run build      # the same build the platform runs
npm run start      # serve it as production

# read the route table it prints:
#   o static      f dynamic      ^ ISR
```

The dev server compiles on demand and tolerates things a production build refuses, so a clean development run tells you almost nothing about whether it will deploy. Read the route table: a page you expected to be static appearing as dynamic usually means something reads cookies or headers.

The pre-flight checklist

CHECK	HOW	IF IT FAILS
The build succeeds	npm run build	Fix it here, not in the build log
Every variable is listed	Compare .env.example	Add it in the platform settings
No secret is public	Search for NEXT_PUBLIC	Rename it and rotate the value
metadataBase is production	Check the root layout	Previews arrive with no image
Remote image hosts allowed	next.config remotePatterns	Images throw at build

Each of those has caused a broken deployment in this module already. Running the list takes less time than reading one platform build log.

What breaks only in production

SYMPTOM	CAUSE
A relative URL fetches nothing	No origin during a server render
Social previews have no image	metadataBase still localhost
A route is unexpectedly dynamic	Something reads cookies
A secret appears in the bundle	NEXT_PUBLIC on the wrong variable
Case-sensitive import fails	macOS forgives, Linux does not

> **Warning The filename one is cruel**
> Importing ./DishCard when the file is ./dishCard works on a Mac and fails on the Linux machine that builds it — it looks like a platform fault and is not. Similarly, fetch("/api/dishes") has no origin during a server render: build the absolute URL from an environment variable, or call the function directly.

Variables, per environment

VARIABLE	DEVELOPMENT	PREVIEW	PRODUCTION
DATABASE_URL	Local database	Seeded test data	The real one
SESSION_SECRET	Anything	A separate value	Strong and unique
NEXT_PUBLIC_SITE_URL	localhost:3000	The preview URL	The live domain
Payment keys	Test keys	Test keys	Live keys

A preview deployment is a real URL on the public internet, so it must never hold production credentials or point at the live database. Remember too that a NEXT_PUBLIC variable is baked in at **build** time — changing one requires a redeploy, not a restart.

Part 4 · Deploying to Vercel

#	STEP
1	Push the project to GitHub
2	Import the repository into Vercel
3	Add every environment variable, per environment
4	Deploy — the framework is detected automatically
5	Verify against the checklist on the live URL
6	Add the URL to your README

Step three is where people fail: nothing warns you about a variable you forgot, so the build succeeds and the application fails at runtime with an error that looks unrelated. Netlify, Cloudflare Pages, Railway or a container on any cloud all work — Vercel is simply the least friction for Next.js, and the principles transfer unchanged.

Preview deployments

WHAT HAPPENS	WHY IT MATTERS
Every branch gets a URL	Review a running application
The URL appears on the pull request	Nobody checks out a branch
It is a production build	The same code path as live
Safe data and test keys	Break it without consequence
Merging promotes to production	One path, every time

Sending someone a working URL is the difference between "looks fine" and a real review, particularly for anyone who does not read code. A preview is also a production build on production infrastructure, which makes it the right place to take the numbers for PERF.md.

After it is live

VERIFY	HOW	LOOKING FOR
It works signed out	A private window	No dependence on your session
Protection holds	The three Day 45 attacks	All three refused
Previews render	Share the URL to yourself	A card with an image
Discovery works	/sitemap.xml, /robots.txt	Real routes, no private ones
It is fast on a phone	Lighthouse, throttled	The numbers for PERF.md

The private window matters more than it sounds. Your own browser holds a session, a cache and possibly a service worker — half of "it works on my machine" is those three, and a private window removes all of them in one keystroke.

When it goes wrong

PROBLEM	FIRST MOVE
The build fails	Read the log from the first error
It built but errors live	Check the runtime logs
A missing variable	Add it, then redeploy
A bad deployment is live	Roll back to the previous one
It works in preview only	Compare the two sets of variables

Every deployment stays addressable, so promoting the previous one takes seconds. **Restore service first, then find the cause** — never debug on a broken production.

Assigned Reading

Required

1. **react.dev — Responding to events** What / why: handlers, propagation, and the event object as React presents it.
2. **Vercel — Projects and deployments** What / why: importing a repository, preview deployments and environment variables.

Optional — go deeper

- **WebAIM — Keyboard accessibility** — focus order, visible focus, and reaching everything without a mouse.
- **MDN — Pointer events** — one set of handlers covering mouse, touch and stylus.

Exercises

Do the keyboard pass with the mouse physically out of reach.

1. Replace every clickable div with a button or a link, and restore visible focus.
2. Reach every function using only Tab, Enter and Escape, and fix what you cannot.
3. Disable submit buttons while pending, and confirm a double tap places one order.
4. Check every touch target is at least 44 pixels and nothing hides behind hover.

5. Run npm run build, read the route table, and work the pre-flight checklist.
6. Deploy to Vercel, setting every variable per environment.
7. Run every post-deployment check on the live URL, in a private window.

Mini-Project · Harden, Then Ship

> **Tip What you build**
> Addis Eats reachable entirely by keyboard, usable on a real phone, with no double submits — and deployed to Vercel with variables set per environment and every post-deployment check passing. The deliverable is a URL somebody else can open, in the README.

Requirements

- No clickable divs — every interactive element is a button or a link.
- Visible focus everywhere, and every function reachable by keyboard alone.
- Forms handled with onSubmit, submit buttons disabled while pending.
- Touch targets of at least 44 pixels, and nothing hidden behind hover.
- A clean npm run build, with the pre-flight checklist worked through.
- Deployed with variables set separately for preview and production.
- Every post-deployment check passing, verified in a private window.

Check yourself

- Can you place an order using only the keyboard, start to finish?
- Does a rapid double tap on Place order create one order or two?
- Does the application work in a private window with no session?
- Is any secret visible in the JavaScript the browser downloads?
- Does a shared link from the live URL render as a card?
- Do you know how to roll back, without looking it up?